Управляющие силы и стаи (Steering + Boids)

Управляющие силы и стаи (Steering + Boids)

Проблема: робот, а не враг

В разделе про ИИ врагов мы научили врага бегать к игроку. Преследование выглядело так:

```
direction = normalize(player.pos - enemy.pos)
enemy.pos += direction * speed * delta_time
```

Работает? Да. Выглядит? Ужасно. Враг мгновенно разворачивается на 180°, двигается строго по прямой, не имеет инерции. Если цель сместилась на пиксель - он дёргается. Если достиг точки - мгновенно останавливается. Робот, а не живое существо.

```
update():
    direction = normalize(target - position)
    position += direction * speed * dt
```

Проблема в том, что мы напрямую задаём позицию. Нет скорости как отдельной величины, нет ускорения, нет плавности. Как это исправить?

Идея: управляющие силы

В 1999 году Крейг Рейнольдс (Craig Reynolds) предложил модель **управляющих сил** (steering behaviors). Идея простая:

1. У агента есть **позиция** и **скорость** (вектор, не скаляр).
2. Каждое поведение вычисляет **желаемую скорость** - куда агент *хочет* двигаться.
3. **Управляющая сила** = `desired_velocity - current_velocity`. Это разница между "хочу" и "могу".
4. Сила ограничена `max_force` - агент не может мгновенно развернуться.

Т.е. вместо "телепортации к цели" мы получаем плавное накопление сил. Агент с инерцией: он разгоняется, тормозит, закладывает дугу при повороте. Ровно как настоящий объект с массой.

Каждое поведение - это одна функция, которая возвращает вектор силы. Мы складываем эти векторы и применяем к скорости. Разные комбинации дают совершенно разное

поведение.

Seek и Flee

Простейшая пара. **Seek** - стремиться к цели. **Flee** - бежать от неё.

Seek: вычисляем вектор от агента к цели, нормализуем, умножаем на `max_speed` - получаем *желаемую* скорость. Управляющая сила = `desired - velocity`.

Flee: то же самое, но вектор направлен *от* цели.

У агента четыре параметра: `pos`, `vel` (векторы), `max_speed`, `max_force` (скаляры). Seek и Flee отличаются только направлением:

```
seek(target):
    desired = normalize(target - pos) * max_speed
    steer = desired - vel                           // <-- the key line
    return clamp_magnitude(steer, max_force)

flee(threat):
    desired = normalize(pos - threat) * max_speed    // direction away from th
    steer = desired - vel
    return clamp_magnitude(steer, max_force)

update(steering, dt):
    vel += steering * dt
    vel = clamp_magnitude(vel, max_speed)
    pos += vel * dt
```

`steer = desired - vel` - вот и вся суть steering behaviors в одной строке. Не "куда идти", а "как изменить текущее движение, чтобы приблизиться к желаемому".

`clamp_magnitude` ограничивает **длину** вектора, не обнуляя его - это принципиально. Без ограничения `max_force` агент мог бы мгновенно развернуться, а без ограничения `max_speed` - разгоняться бесконечно. В C# удобно использовать `Vector2` из MonoGame (`Vector2.Normalize`, `Length()`), в Python - `math.hypot` и ручная арифметика с кортежами/списками.

Arrive

Seek работает, но у него есть проблема: агент на полной скорости влетает в цель, проскакивает, разворачивается, проскакивает обратно - бесконечные колебания. Он никогда не "остановится", потому что `desired` всегда направлено к цели с полной скоростью.

Arrive решает это: когда агент входит в **радиус замедления** (`slow_radius`), скорость пропорционально уменьшается. Чем ближе к цели - тем медленнее.

```
arrive(target, slow_radius=100):
    dist = distance(pos, target)
    if dist < slow_radius:
        speed = max_speed * (dist / slow_radius)   // linear deceleration
    else:
        speed = max_speed

    desired = normalize(target - pos) * speed
    steer = desired - vel
    return clamp_magnitude(steer, max_force)
```

Грубо говоря, Arrive = Seek, но с `max_speed`, масштабированным по расстоянию до цели. Вне `slow_radius` - полная скорость. Внутри - скорость пропорциональна оставшемуся расстоянию. На расстоянии 10 при `slow_radius=100` скорость = 10% от максимальной. Результат: агент плавно подходит и останавливается, вместо того чтобы пролетать мимо и метаться.

Wander

Seek и Flee целенаправленны - агент знает, куда идти. А что, если ему нечем заняться? Нужно бесцельное блуждание, но *не* случайные дёрганья.

Идея Рейнольдса: проецируем **окружность** перед агентом (на расстоянии `wander_distance`). На этой окружности выбираем точку с углом `wander_angle`, который каждый кадр немного смещается на случайную величину. Затем - обычный Seek к этой точке:

```
wander():
    circle_center = pos + normalize(vel) * wander_distance
    wander_angle += random(-0.3, 0.3)   // small random nudge each frame
    target = circle_center + (cos(wander_angle), sin(wander_angle)) * wander_ra
    return seek(target)
```

Поскольку угол меняется плавно (±0.3 радиан за кадр), цель на окружности сдвигается понемногу - и агент плавно поворачивает, как муха.

Pursuit и Evade

Seek нацеливается на текущую позицию цели. Но если цель движется, агент вечно "опаздывает" - бежит туда, где цель была, а не туда, где она будет.

Pursuit предсказывает будущую позицию цели: `future = target.pos + target.vel × T`, где `T` - время "упреждения". Чем дальше цель, тем больше `T`. Затем - обычный Seek к предсказанной точке.

Evade - то же, но с Flee от предсказанной позиции.

```
pursuit(target_agent):
    dist = distance(pos, target_agent.pos)
    T = min(dist / max_speed, max_prediction)    // farther target = longer pr
    future = target_agent.pos + target_agent.vel * T
    return seek(future)

evade(threat_agent):
    // same thing, but flee instead of seek
    return flee(future)
```

Pursuit заметно умнее Seek: если цель бежит вправо, агент срежет угол, а не побежит точно по следу. Для вражеских ракет, преследующих хищников, перехватчиков - Pursuit обязателен.

Комбинирование сил

Каждое поведение возвращает вектор. Но как их смешать? Допустим, враг должен одновременно стремиться к игроку (Seek) и избегать стен (Flee от ближайшей стены). Два вектора - как их сложить?

Взвешенная сумма - самый простой способ:

```
total = seek_force * w_seek + flee_force * w_flee + wander_force * w_wander
total = clamp(total, max_force)
```

Веса определяют приоритет. Конкретный пример: враг бежит к игроку и избегает стен. Для каждой стены в радиусе 80 считаем `flee`, взвешиваем обратно пропорционально расстоянию (ближе = сильнее), и суммируем с `seek`:

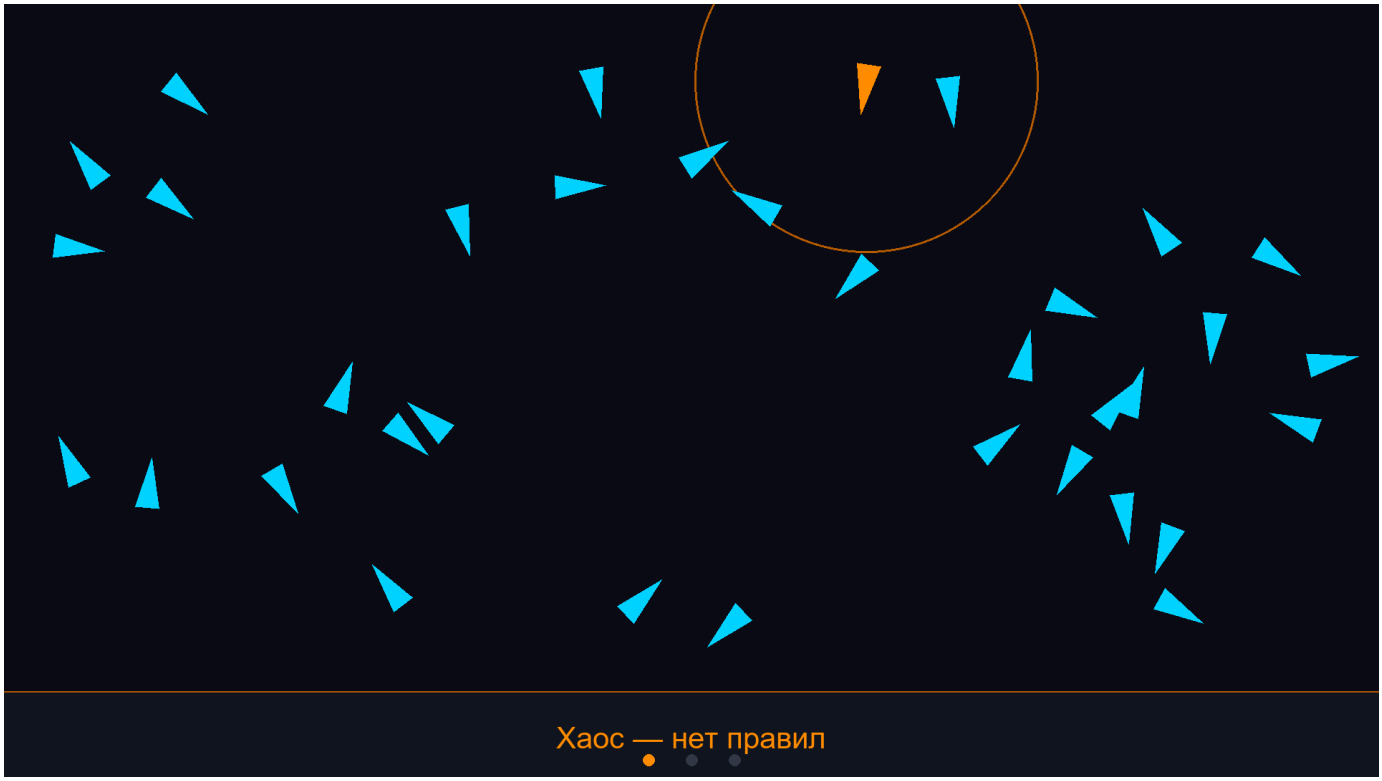
```
seek_f = seek(target) * 1.0
avoid_f = sum(flee(wall) * (1 - dist/80) for wall in nearby_walls) * 2.0
total = clamp_magnitude(seek_f + avoid_f, max_force)
```

Вес 2.0 у избегания означает: "стены важнее цели". Хотите более агрессивного врага, который рискует? Снизьте вес `avoid`.

Есть и другие стратегии: **приоритетное отсечение** (priority truncation - применяем силы по приоритету, пока не исчерпаем `max_force`) и **взвешенная сумма с нормализацией**. Для студенческого проекта простая взвешенная сумма работает отлично.

> ⚠ В разделе про порядок обновления мы обсуждали, что важно применять силы с учётом `deltaTime`. Управляющие силы - не исключение: `vel += force * dt`, `pos += vel * dt`.

Стая: Boids



Ок, одиночный агент двигается плавно. А что, если их 30? 100? Как сделать стаю летучих мышей, косяк рыб, толпу зомби?

В 1987 году тот же Рейнольдс показал, что сложное стайное поведение возникает из трёх простых правил. Каждый агент (boid) смотрит на **соседей в определённом радиусе** и вычисляет три силы:

1. **Separation** (разделение) - держать дистанцию. Стремиться прочь от слишком близких соседей. Вес обратно пропорционален расстоянию: чем ближе сосед, тем сильнее отталкивание.
2. **Alignment** (выравнивание) - двигаться как соседи. Стремиться к средней скорости соседей. Это создаёт согласованное направление стаи.
3. **Cohesion** (сплочённость) - держаться вместе. Стремиться к центру масс соседей. Это не даёт стае "рассыпаться".

Каждая функция принимает список соседей в радиусе и возвращает вектор силы. Паттерн одинаковый: пройтись по соседям, агрегировать что-то, превратить результат в steering force (`desired - vel`):

```
separation(neighbors, radius=40):
    // for each neighbor: vector "from it to us", weighted by 1/dist
    // average, normalize * max_speed, subtract vel
    // closer neighbor = stronger repulsion

alignment(neighbors, radius=80):
    // average velocity of neighbors
    // normalize * max_speed, subtract vel
    // "move like everyone around me"


```

```
cohesion(neighbors, radius=80):
    // center of mass of neighbors
    // call seek(centroid) - yes, just reuse seek!
    // "stay close to the group"

flock(neighbors):
    sep = separation(neighbors) * 1.5   // priority: don't step on each other
    ali = alignment(neighbors)  * 1.0
    coh = cohesion(neighbors)   * 1.0
    return clamp_magnitude(sep + ali + coh, max_force)
```

Три функции, каждая возвращает вектор. `flock` складывает их с весами и ограничивает. Вот и вся "стая".

Одни параметры - разные стаи

Те же три правила, разные веса и радиусы - совершенно разное поведение. Тот же код, другие числа:

Параметр	Косяк рыб	Стая птиц	Толпа зомби
Separation вес	1.0	1.2	2.0
Separation радиус	25	40	50
Alignment вес	1.5	1.0	0.3
Alignment радиус	60	100	30
Cohesion вес	2.0	0.8	0.5
Cohesion радиус	80	120	60
Max speed	100	200	60

Рыбы: сильная сплочённость, плотный косяк, согласованные повороты. Птицы: широкое выравнивание, рыхлая стая, высокая скорость. Зомби: сильное разделение (не наступают друг на друга), слабая координация, медленные.

Это та же идея, что и в разделе про ИИ врагов: одна модель, разные параметры - разные "личности". Steering behaviors доводят этот принцип до максимума.

Оптимизация: O(n²) -> пространственные структуры

Каждый boid проверяет каждого другого boid - это $O(n^2)$. Для 40 агентов: 40 × 39 = 1560 проверок каждый кадр. Терпимо. Для 400: 400 × 399 = 159 600. Для 1000: почти миллион. На каждый кадр.

Нетерпимо.

Но нам ведь не нужны *все* соседи - только те, кто в пределах радиуса (40-80 пикселей). А это классическая задача **пространственного поиска**.

Python C#

```python
# naive: O(n^2)
for boid in boids:
    neighbors = [b for b in boids if b is not boid
                 and distance(boid, b) < radius]
    steering = boid.flock(neighbors)

# with uniform grid: O(n * k), where k = neighbors per cell
from spatial_grid import UniformGrid

grid = UniformGrid(cell_size=100, width=800, height=600)
for boid in boids:
    grid.insert(boid, boid.pos[0], boid.pos[1])

for boid in boids:
    nearby = grid.query(boid.pos[0], boid.pos[1], radius=80)
    neighbors = [b for b in nearby if b is not boid]
    steering = boid.flock(neighbors)
```

Подробнее про Uniform Grid, QuadTree и пространственный хеш: Пространственные структуры данных. Для Boids обычно хватает простой сетки - агенты распределены относительно равномерно.

Ограничения

Steering behaviors - не серебряная пуля. Чисто реактивные силы не умеют планировать: агент с Seek упрётся в U-образную стену и застрянет, потому что силы отталкивания уравновесятся. Для навигации в сложном окружении нужен A*, а steering отвечает за *как* двигаться к очередной точке маршрута.

Ещё один подвох - подбор весов и радиусов. Таблица выше выглядит аккуратно, но на практике хорошие значения ищутся перебором: чуть перекрутил `separation` - стая разлетается, занизил `cohesion` - рассыпается. Грубо говоря, steering даёт красивую модель, но её настройка - ручная работа.

Больше, чем игровой трюк

Управляющие силы - это больше, чем игровой трюк. За ними стоят серьёзные концепции:

- **Реактивные агенты.** Каждый boid реагирует только на локальное окружение, не имея глобального плана. Это идея подчинённой архитектуры (subsumption architecture) Родни Брукса - интеллект как набор простых слоёв, реагирующих на среду.
- **Потенциальные поля.** Seek/Flee - это, по сути, градиентный спуск по полю притяжения/отталкивания. Та же математика используется в робототехнике для планирования движения.
- **Роевой интеллект.** Boids - частный случай swarm intelligence. Из простых локальных правил возникает сложное глобальное поведение. Тот же принцип лежит в основе муравьиных алгоритмов (ACO) и роя частиц (PSO) - методов оптимизации, которые вы встретите в курсе ML.
- **Статья Рейнольдса (1987).** "Flocks, Herds, and Schools: A Distributed Behavioral Model" - одна из самых цитируемых работ в Computer Graphics. Стоит прочитать оригинал.

Steering behaviors - мост между конечными автоматами (дискретные состояния) и непрерывным управлением. Автомат решает *что* делать (патрулировать, преследовать), а steering - *как* двигаться (плавно, с инерцией, с уклонением).

Итого

Что	Где подробнее
FSM врага (Patrol -> Chase -> Attack)	Простой ИИ врагов
Конечные автоматы для выбора поведения	Конечные автоматы
A* для навигации между steering-целями	Поиск путей
Grid / QuadTree для O(n) neighbor query	Пространственные структуры
Деревья поведений для сложного ИИ поверх steering	Деревья поведений
Проверка столкновений с окружением	Коллизии
deltaTime и порядок обновления сил	Порядок обновления
Частицы - похожая архитектура (много объектов, простые правила)	Системы частиц
Сложность алгоритмов и обзор	Алгоритмы

Powered by Wiki.js